

Improving Reliability of Real-Time RTP Audio Streaming Under Packet Loss Using FEC, Adaptive Jitter Buffering, and Waveform-Substitution PLC

Devika Sana

Department of Computer Science and Engineering
Rajiv Gandhi University of Knowledge and Technologies
sanadevika23@gmail.com

Abstract—Real-time audio streaming over UDP is widely used because UDP avoids the delay penalties of retransmission-based protocols such as TCP. However, UDP does not guarantee delivery, and packet loss on the network directly translates into audible glitches, dropouts, and interruptions. This paper presents a practical, undergraduate-level study of a real-time audio streaming system built using the Real-time Transport Protocol (RTP), FLAC audio compression, and ALSA-based playback, implemented in C++ using `jrtplib3`. The existing system streams 16-bit PCM audio from a WAV file through a splitter, a FLAC-encoding RTP client, and an RTP server that decodes and plays the audio. To study the effect of network unreliability, packet loss was manually introduced at the sender at six configured levels (0%, 1%, 5%, 10%, 20%, and 50%), and the actual loss percentage and resulting audio quality were recorded over 11,437 transmitted RTP packets.

The results confirm that even small amounts of loss are audible and that loss above 10% makes playback significantly degraded. Three practical improvements—XOR-based Forward Error Correction, an adaptive jitter buffer, and waveform-substitution packet loss concealment—were then added to the same codebase and evaluated under the same loss conditions. The improved system does not eliminate the effect of packet loss but noticeably reduces audible glitches and improves playback continuity at low-to-moderate loss rates, while still degrading at very high loss rates. This paper documents the baseline measurements, the design and integration of the improvements into the existing C++ files, and a realistic comparison between the two versions of the system.

Index Terms—RTP, RTCP, Packet Loss, Forward Error Correction, Jitter Buffer, Packet Loss Concealment, FLAC, Real-Time Audio Streaming.

I. INTRODUCTION

Real-time audio applications such as voice calls, live streaming, and intercom systems depend on transporting audio data over a network with minimal delay. The Real-time Transport Protocol (RTP), typically running over UDP, is the standard choice for such applications because UDP does not introduce the retransmission and ordering delays associated with TCP. This makes UDP suitable for time-sensitive media, but it comes at a cost: UDP provides no guarantee that a packet will arrive, and no automatic mechanism to recover a packet that is lost in transit.

For real-time audio, packet loss is a serious problem. Each RTP packet in a typical streaming pipeline carries a short segment of audio (in this project, 1024 PCM samples per

channel, roughly 23 milliseconds of audio at 44.1 kHz). When a packet is lost, that segment of audio is simply missing at the receiver. Because audio is a continuous waveform, an abrupt gap is perceived by the listener as a click, glitch, or dropout, even if the missing segment is only a few milliseconds long. As the rate of packet loss increases, these gaps become more frequent and more noticeable, eventually making the audio difficult to understand or unpleasant to listen to.

The objective of this paper is twofold. First, it documents a working RTP-based audio streaming system implemented in C++, and quantifies how its playback quality degrades as packet loss increases, using measurements collected by manually simulating loss at the sender. Second, it describes three practical, well-established techniques—Forward Error Correction (FEC), adaptive jitter buffering, and waveform-substitution packet loss concealment (PLC)—that were added to the existing codebase to reduce the impact of packet loss, and presents a realistic before-and-after comparison. This paper does not propose a new protocol or a new algorithm; it applies existing, well-known techniques to an existing student project and reports the observed results.

II. RELATED WORK

RTP and its companion control protocol RTCP were originally standardized to support the transport and monitoring of real-time media over packet networks and remain the basis for most modern real-time audio and video systems. RTP itself does not solve packet loss; it only provides sequence numbers and timestamps that allow a receiver to detect loss, reordering, and timing information, leaving loss recovery and concealment to the application.

Forward Error Correction is a long-established technique for combating packet loss in real-time media without requiring retransmission. In its simplest form, redundant parity data is generated from a group of original packets using XOR operations and sent alongside them. If one packet in the group is lost, the parity packet can reconstruct it without adding round-trip delay.

Jitter buffering is another standard component of RTP receivers. Since network delay is not constant, packets do not always arrive in the same order or with the same spacing that they were sent. A jitter buffer temporarily stores incoming

packets, reorders them according to RTP sequence numbers, and releases them for playback at a steady rate.

Packet Loss Concealment (PLC) is used when a packet is missing and cannot be recovered through FEC. Instead of inserting silence, the decoder approximates the missing audio by replaying or fading the previous frame, making the interruption less noticeable.

Finally, this project uses the FLAC (Free Lossless Audio Codec) format to compress each PCM frame before transmission. Although FLAC is not specifically designed for real-time streaming, each frame is encoded independently, allowing it to fit naturally into a frame-per-packet RTP architecture.

III. EXISTING SYSTEM

The proposed work is based on an existing C++ RTP audio streaming pipeline consisting of three programs:

WAV File → Splitter → FIFO → Client (FLAC Encode) → RTP → Server (FLAC Decode) → Speaker

A. *splitter.cpp*

This program reads a 16-bit PCM WAV file and divides it into fixed-size frames of 1024 samples per channel. Instead of writing temporary PCM files, it creates a named FIFO using `mkfifo()` and streams each frame directly to the client process as a length-prefixed block.

B. *client.cpp*

The client reads PCM frames from the FIFO, independently FLAC-encodes each frame, and transmits it as a single RTP packet using `jrtplib3` with payload type 96. Packet transmission is paced in real time by introducing approximately 23.2 ms delay between packets.

C. *server.cpp*

The server receives RTP packets, decodes the FLAC payload into PCM samples, plays the decoded audio using ALSA, and simultaneously stores the decoded PCM into `decoded.pcm` for verification.

D. *System Parameters*

TABLE I
SYSTEM PARAMETERS

Parameter	Value
Sample Rate	44,100 Hz
Channels	Stereo (2)
Frame Size	1024 samples/channel
Audio Format	16-bit PCM
Transport	RTP over UDP
Payload Type	96
Compression	FLAC
Playback	ALSA
Frame Duration	23.2 ms

TABLE II
BASELINE PACKET-LOSS MEASUREMENTS

Configured Loss	Packets Lost	Actual Loss (%)	Audio Quality
0%	0	0.0000	Perfect audio
1%	114	0.9967	Almost unnoticeable
5%	574	5.0188	Noticeable glitches
10%	1149	10.0463	Frequent interruptions
20%	2299	20.1041	Very noticeable degradation
50%	5737	50.1618	Audio nearly unusable

IV. BASELINE PACKET-LOSS EXPERIMENT

To study how the existing system behaves under unreliable network conditions, packet loss was manually introduced at the sender side (in `client.cpp`, before packets were handed to `jrtplib3` for transmission). A fixed audio file was streamed through the system at six different configured loss levels, and the actual number of packets lost, the resulting actual loss percentage, and the subjective audio quality were recorded by listening to the playback at the server.

A total of 11,437 RTP packets were transmitted across the experiment. These values were obtained through manual measurement during experimentation rather than simulation.

The experimental results indicate that the actual measured packet-loss percentage closely matched the configured values. Audio quality gradually deteriorated as packet loss increased. At approximately 1% loss, playback remained almost unaffected, whereas losses above 10% resulted in frequent interruptions. At 50% packet loss, the received audio became nearly unusable.

V. PROPOSED IMPROVEMENTS

To improve playback quality under packet loss, three practical techniques were integrated into the existing RTP streaming system.

A. *XOR-Based Forward Error Correction (FEC)*

Packets are grouped into sets of four RTP packets. For every group, one additional parity packet is generated by XOR-ing the payload bytes of all four packets.

If exactly one packet within a group is lost, the receiver reconstructs it using the remaining packets together with the parity packet.

The main advantage of XOR-based FEC is that packet recovery occurs without retransmission, making it suitable for real-time communication.

The drawback is a bandwidth overhead of approximately 25%, since one parity packet is transmitted for every four original packets.

B. *Adaptive Jitter Buffer*

Incoming RTP packets are first stored inside a jitter buffer instead of being decoded immediately.

Packets are reordered according to their RTP sequence numbers, allowing slightly delayed packets to arrive before playback.

The waiting time of the jitter buffer automatically increases or decreases depending on the measured network jitter.

This approach reduces false packet-loss detection caused by packet reordering or variable network delay.

C. Waveform-Substitution Packet Loss Concealment

If a packet cannot be recovered using FEC, the receiver applies Packet Loss Concealment (PLC).

Instead of inserting silence, the previously decoded PCM frame is replayed with a gradual fade-out effect.

Although PLC does not reconstruct the original audio, it significantly reduces audible clicks and makes missing packets less noticeable.

VI. INTEGRATION WITH EXISTING CODE

The proposed techniques were integrated into the existing three-file C++ implementation.

A. *splitter.cpp*

No modifications were required.

The splitter only reads the WAV file and streams PCM frames into the FIFO.

Since FEC, jitter buffering, and PLC operate after encoding and during network transmission, the splitter remains unchanged.

B. *client.cpp*

The client was modified to generate XOR parity packets.

After every four FLAC-encoded RTP packets, one parity packet is generated and transmitted.

The RTP sequence numbering was preserved to allow the receiver to correctly identify packet groups.

C. *server.cpp*

The server performs three additional operations before playback:

- 1) Store incoming packets inside the adaptive jitter buffer.
- 2) Recover a missing packet using XOR-based FEC whenever possible.
- 3) Apply waveform-substitution PLC if packet recovery fails.

Recovered or concealed PCM samples are finally decoded and played through ALSA exactly as in the original implementation.

The original RTP streaming pipeline remains unchanged while additional recovery and concealment mechanisms improve robustness under packet loss.

VII. RESULTS AND COMPARISON

The improved system was evaluated under the same six configured packet-loss levels used in the baseline experiment. Audio quality was assessed through subjective listening tests together with an approximate Mean Opinion Score (MOS) on a scale from 1 to 5.

The proposed techniques consistently improved playback quality at low and moderate packet-loss levels. The greatest improvement was observed between 1% and 10% packet loss, where most packet losses occurred individually and could therefore be recovered by XOR-based FEC.

As packet loss increased beyond 20%, the probability of losing multiple packets within the same FEC group increased, reducing the recovery capability. Packet Loss Concealment continued to reduce audible clicks and silence but could not reconstruct the original missing audio.

VIII. DISCUSSION

Each proposed technique contributes differently toward improving playback quality.

Forward Error Correction recovers isolated packet losses without requiring retransmission, making it highly suitable for real-time RTP applications. However, its recovery capability is limited to one lost packet per FEC group.

The adaptive jitter buffer reduces false packet-loss detection by compensating for packet reordering and variable network delay. This improves playback continuity while introducing only a small additional delay.

Packet Loss Concealment provides the final layer of protection by replacing unrecoverable packets with a faded repetition of the previous audio frame. Although this does not restore the original audio information, it significantly reduces perceptible glitches.

Combining all three techniques provides better overall robustness than using any individual technique alone. The improvement is particularly significant under realistic network conditions where packet loss is generally below 10%.

IX. CONCLUSION

This paper presented a practical enhancement to an existing RTP-based real-time audio streaming system implemented in C++. The original implementation successfully streamed FLAC-compressed audio over RTP but suffered noticeable degradation under packet-loss conditions.

Three well-established techniques—XOR-based Forward Error Correction, adaptive jitter buffering, and waveform-substitution Packet Loss Concealment—were integrated into the existing implementation without changing its overall architecture.

Experimental evaluation using six packet-loss levels demonstrated that the proposed improvements significantly enhanced playback continuity under low and moderate packet-loss conditions while providing limited but noticeable improvements even under severe packet loss.

The proposed system therefore represents a practical and lightweight enhancement for real-time RTP audio streaming applications.

REFERENCES

- [1] H. Schulzrinne, S. Casner, R. Frederick, and V. Jacobson, "RTP: A Transport Protocol for Real-Time Applications," RFC 3550, IETF, 2003.
- [2] H. Schulzrinne and S. Casner, "RTP Profile for Audio and Video Conferences with Minimal Control," RFC 3551, IETF, 2003.
- [3] C. Perkins, O. Hodson, and V. Hardman, "A Survey of Packet Loss Recovery Techniques for Streaming Audio," IEEE Network, vol. 12, no. 5, pp. 40–48, 1998.
- [4] J. C. Bolot, S. Fosse-Parisis, and D. Towsley, "Adaptive FEC-Based Error Control for Internet Telephony," Proc. IEEE INFOCOM, 1999.
- [5] B. Goode, "Voice over Internet Protocol (VoIP)," Proceedings of the IEEE, vol. 90, no. 9, pp. 1495–1517, 2002.

TABLE III
COMPARISON BETWEEN EXISTING AND IMPROVED RTP SYSTEM

Configured Loss	Existing System (Approx. MOS)	Improved System (Approx. MOS)	Improvement
0%	Perfect audio (5.0)	Perfect audio (5.0)	No packet loss, therefore no difference.
1%	Almost unnoticeable (4.6)	Effectively unnoticeable (4.8)	Most isolated packet losses recovered by FEC.
5%	Noticeable glitches (3.6)	Minor occasional glitches (4.2)	Most single packet losses recovered successfully.
10%	Frequent interruptions (2.7)	Mostly continuous playback (3.5)	FEC recovery combined with PLC noticeably improves listening quality.
20%	Very noticeable degradation (1.9)	Still degraded but more listenable (2.6)	FEC recovery becomes less effective as multiple packet losses occur.
50%	Nearly unusable (1.1)	Still heavily degraded (1.6)	PLC softens missing audio but cannot restore intelligibility.

- [6] R. Ramjee, J. Kurose, D. Towsley, and H. Schulzrinne, "Adaptive Play-out Mechanisms for Packetized Audio Applications," IEEE INFOCOM, 1994.
- [7] Y. J. Liang, N. Farber, and B. Girod, "Adaptive Playout Scheduling and Loss Concealment for Voice Communication over IP Networks," IEEE Transactions on Multimedia, vol. 5, no. 4, pp. 532–543, 2003.
- [8] C. Perkins and O. Hodson, "Options for Repair of Streaming Media," RFC 2354, IETF, 1998.
- [9] Xiph.Org Foundation, "FLAC: Free Lossless Audio Codec."
- [10] Advanced Linux Sound Architecture (ALSA), "ALSA Library API Documentation."
- [11] JRTPLIB Project, "An Object-Oriented RTP Library Written in C++."
- [12] C. Boutremans, G. Iannaccone, and C. Diot, "Impact of Link Failures on VoIP Performance," NOSSDAV, 2002.